A STEP-BY-STEP GUIDE

From idea to a working proof of concept on Azure

For people who know their subject well and want to test an idea with a real system before committing to a full project.

Start with Part 1

PART 1
What a POC is
The question, the scope, the people.

PART 2
Ideation
Find the real problem with the people who live with it.

PART 3
Choose and cost
Compare options, price the one you pick.

PART 4
How it fits together
Who does what inside the system.

PART 5
Getting access
Why waiting for permission slows most POCs.

PART 6
Building it
Seven steps, foundations first.

PART 7
Putting it on Azure
First a quick demo, then the secure version.

PART 8
Keeping it safe
Habits that let a POC grow up safely.

PART 9
Show it and decide
Demo, measure, decision paper.

PART 10
When things go wrong
Common problems and what to do.

PART 11
Checklist and plan
Definition of done and a four-week plan.

Two ways to read

Use **Plain language** for the ideas. Switch on + **Technical detail** (top right) to add commands and settings for whoever builds it.

One running example

LeaveLoop: a made-up tool for requesting and approving time off. Nothing real or confidential appears here.

Figures are rounded planning estimates, not quotes. Check services, limits and prices in Microsoft's documentation and the Azure pricing calculator before committing.

PART 1

What a proof of concept is

A POC answers one question: **is this idea worth investing in?** It is not a small version of the finished product.

POC, pilot, production

POC

Weeks, fake data. Built to be judged, then kept or dropped.

Pilot

Real users and data, limited scope, with support. Only after a POC says yes.

Production

Everyone, full security, an owner on call. A project, not an experiment.

THE RULE THAT KEEPS IT SMALL

If a feature doesn't help answer the question, it goes on the "not now" list.

Six decisions

Write these down before any work starts.

Decide	Example (LeaveLoop)
The question	"Can leave be handled online faster, with a clear record of who approved what?"
Users and roles	Employee, manager, HR viewer
In / out of scope	In: request, approve, summary. Out: payroll link, mobile app
Success measures	Time from request to decision; 4 of 5 testers would switch
Time box	3 weeks to build, 1 to test
Data	Invented employees and requests only

Who's involved

Person	Role	LeaveLoop
Sponsor	Owns the final decision; clears blockers	Head of HR
Subject lead (you)	Knows the process; gathers insight; judges the result	HR operations lead
Builder	Creates and deploys it	One developer, part time
Cloud / IT team	Grants access; reviews security	Central IT
Real users	Try it and tell you the truth	5 employees, 3 managers

PART 2

Ideation: find the right problem

Most failed POCs solve the wrong problem. **Listen first, define second, build third.**

Who to talk to

The doers

Know the real steps. *HR admins.*

The users of the output

Show what's actually needed. *Managers.*

The owner

Defines success and limits. *Head of HR.*

IT and risk

Can stop you late if skipped early. *Data protection, cloud team.*

Aim for **6 to 10 conversations** of 30 minutes. After about five on one theme you mostly hear repeats.

Interview well

- **Ask about the last time,** not "usually". Facts beat tidy stories.
- **Listen for workarounds.** A personal spreadsheet is an unwritten requirement.
- **Ask to see the real files.**
- **Don't pitch your idea.** People then answer politely, not honestly.
- **Two people:** one asks, one writes. **Play it back** within a day: "what did I get wrong?"

Question bank

1. Walk me through the last time you did this, start to finish.
2. Which step is slowest? Which do you dread?
3. Where do mistakes happen, and what did the last one cost?
4. What do you keep outside the official system, and why?

1. What decision do you make with this? When do you need it?
2. What do you do when it's late or looks wrong?
3. What would make you trust it more?
4. What must never be shown to others?

1. What would make you say "worth it" in six months?
2. What does the problem cost today (time, money, risk)?
3. What must not change? Who else must be on board?
4. What would make you stop?

1. What data may a POC hold, and where?
2. Which regions, tags and naming rules are mandatory?
3. Which sign-in method must we use?
4. Who owns the system after the POC?

Make sense of it

1 Five whys

Ask "why?" until you reach a cause you can act on.

2 Map the journey

One row per step: who, pain, evidence (a quote or number).

3 "How might we..."

Turn each pain into an open question.

4 One-sentence problem

Test it on two interviewees. If they don't nod, keep going.

LEAVELOOP, FIVE WHYS

Requests are approved late → managers don't see them → they sit among hundreds of emails → no single list of what's waiting. **The problem is visibility, not approval rules.**

Step today	Pain	Evidence
Employee emails manager	No confirmation it arrived	"I ask in the corridor"
HR retypes into a master sheet	Clashing versions	Two files called "final"

Design thinking

Start with people, test cheaply, repeat. Five stages, and you loop back often.

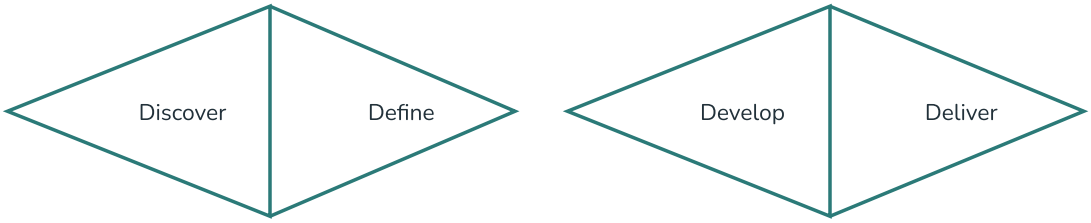
1 Empathise → 2 Define → 3 Ideate → 4 Prototype → 5 Test

Double diamond

Open up, then narrow, twice: first to find the *right problem*, then the *right solution*.

Jobs to be done

Ask why someone would use it. Managers don't want a leave system; they want to *plan cover without chasing people*.



Break it down

Roles

Role	Sees	Changes
Employee	Own requests	Create, withdraw
Manager	Team's requests	Approve, decline
HR viewer	Summaries	Nothing

Workflow as states

Draft → Submitted → Approved or Declined

For each arrow, write who may make the move. These small tables become the system's rule book.

Must / should / could / won't

Must

- Submit
- Approve or decline
- History

Should

- Reminders
- HR summary

Could

- Team calendar

Won't (now)

- Payroll, mobile

BUILD THE THIN SLICE FIRST

The smallest complete path: *employee submits → manager approves → HR sees it.* It proves the idea and always leaves you something to show.

Prototype and test

Paper or slides

Sketch screens; walk someone through.

Clickable mock-up

Linked slides. Good for layout and wording.

"Wizard of Oz"

Looks real; a person does the work behind it. Tests the idea before the engine.

Working thin slice

The real thing, minimal, with fake data.

- Test with **3 to 5 people**. Give a task, then stay quiet.
- Where they hesitate, the design needs work.
- Write findings in their own words.

Work as a team

- **One-page brief** everyone has seen (template below).
- **Weekly 20-minute show-and-tell.** Show what works; no slides.
- **Decision log:** date, decision, why, who. Ends "I thought we agreed...".
- **Say "that's for later" kindly:** "Great idea. It's on the next-version list."
- **Agree who decides** scope changes and signs off the demo.

```
POC BRIEF (one page)
Question this POC answers:
Users and roles:
Workflow (states, who moves them):
Data (what we store; fake or real):
Must / Should / Could / Won't:
Success measures:
Time box and key dates:
Decision owner:
Risks and open questions:
```

TECHNICAL DETAIL

From brief to data model

Nouns become tables; states become a rule table. LeaveLoop: `employees`, `leave_requests` (status, decided_by, decided_at, reason), `audit_log`.

- **Key by what makes a record unique**, usually unit *and* period, or next year overwrites this one.
- **Constraints in the database** (`CHECK`, `UNIQUE`, foreign keys) catch what code misses.
- **Rule table first:** `(from, to) → roles` and `state → role → fields`. Enforce on the server.
- **Review the schema with a human before any screen.** It's the costliest thing to change later.

PART 3

Choose the approach, and price it

Check that building is the right answer. Then pick the **simplest design** that answers the question.

The options

Option	Good when	Watch out for
Improve what you have	Pain is habit or clarity	Ceiling on history, permissions
Buy a product	A tool fits ~80%	Per-user cost; hard to bend
Low-code	Simple, quick need	Limits at scale
Build on Azure	Specific process; need control	You maintain it
Do nothing (for now)	Small problem, wrong timing	Cost of waiting

Compare fairly

Weight each criterion 1 to 5, score each option 1 to 5. **The example scores are placeholders, deliberately close.** Do it with two or three colleagues and discuss where you disagree.

Decision matrix

Criterion	Weight	Improve	Buy a pl	Low-coc	Build or
Fit to our process	5	2	3	3	5
Time to first value	4	5	3	4	2
12-month cost (incl. people	4	5	2	3	2
Security and data control	4	2	4	3	4
Skills we already have	3	5	4	3	2
Ease of changing later	3	2	2	3	5
Weighted score		**79 (69%)**	**69 (60%)**	**73 (63%)**	**78 (68%)**

Names are editable. Nothing leaves your browser.

What it costs

PEOPLE ARE THE BIGGEST COST

A few weeks of a developer's and a subject lead's time usually cost many times the cloud bill. Include it.

What drives the Azure bill

- Hosting plan size (billed even if idle)
- Database size
- Security extras (firewalls, private networking, geo-backups)
- Number of environments
- Forgotten resources

Guardrails

- Budget alert on day one (50 / 80 / 100%)
- Tag everything with owner and cost centre
- Diary a shut-down review date
- One person owns the bill

Rough monthly cost estimator

Rounded US$ planning figures, **not a quote**. Confirm in the Azure pricing calculator.

API hosting (App Service)	Front end (Static Web Apps)	Database
Basic B1 (~$13)	Free	File database on web app (d

File storage (Blob)	Key Vault	Monitoring
None	None (settings only)	None

Edge protection	Private networking, threat protection
None	None

Environments	Build and support hours	Cost per hour (US$)
1	160	60

POC length (months)

3

$13	$39	$9,600	$9,639
cloud cost per month	cloud cost over 3 months	people time (one-off)	total POC cost

People time is about 246 times the cloud cost here. Rounded planning estimates, not quotes.

TECHNICAL DETAIL

- Use the Azure pricing calculator with the right region; always-on is about 730 hours a month.
- Biggest to smallest for this stack: App Service plan, SQL database, edge/WAF or private networking, monitoring ingestion. Storage and Key Vault are small.
- Budget and tags: Cost Management in the portal; `--tags CostCenter=... Owner=...` at creation.
- Log Analytics bills by data ingested; keep retention short for a POC.

Azure choices

Choose the simplest service now; move up when evidence says so.

Decision	Start with	Move up when
Screens	Static Web Apps (free)	You need a gateway or server-rendered pages
Logic	App Service, one small plan	Containers for many services; Functions for events
Data	Demo: a file database. Then Azure SQL	PostgreSQL by policy; Cosmos DB for flexible or global data
Files	Private Blob Storage	Immutable storage for records
Sign-in	Simple login (demo only)	Entra ID before real data
Secrets	App settings (fake data only)	Key Vault before real data

Use AI to help

Good for

- Options you missed
- Drafting criteria and weights
- Explaining pricing pages
- A first cost model, assumptions listed
- "What would make this fail?"
- Themes from interview notes

Be careful

- **Prices and limits:** can be confidently wrong. Verify in the calculator or with IT.
- **Confidential data:** never paste it.
- **Authority:** it suggests; your team decides.

Copy-and-adapt prompts

```
I need to [one-sentence problem] for about [number] users in [type of organisation].
Constraints: [budget, timeline, data sensitivity, skills].
List 5 realistic approaches (including improving what we have and doing nothing).
For each: when it fits, main risks, and what a 4-week test would look like.
Don't recommend one yet. Mark anything you're unsure about.
```

```
We're choosing between: [options].
Propose 8 criteria for a proof of concept (not production), each with a weight 1 to 5
and what a score of 1, 3 and 5 means. Flag overlaps and anything I'm missing.
```

```
Estimate the monthly cost of this design on Microsoft Azure:
[front end, API host, database, storage, monitoring, security extras], region [region],
about [n] users, [n] environments.
```

```
List every assumption, give low / expected / high, and say which numbers I must verify
in the Azure pricing calculator. Don't present any price as final.
```

```
Here is our plan and why we chose it: [paste].
Act as a sceptical reviewer. Give the 5 strongest reasons it could fail or be the wrong
choice,
the evidence that would change your mind, and the cheapest test for each.
```

```
Below are anonymised notes from [n] interviews (no names, no confidential data):
[paste]
Group them into themes with exact supporting quotes, note where people disagree,
and list questions I should have asked.
```

GOOD HABIT

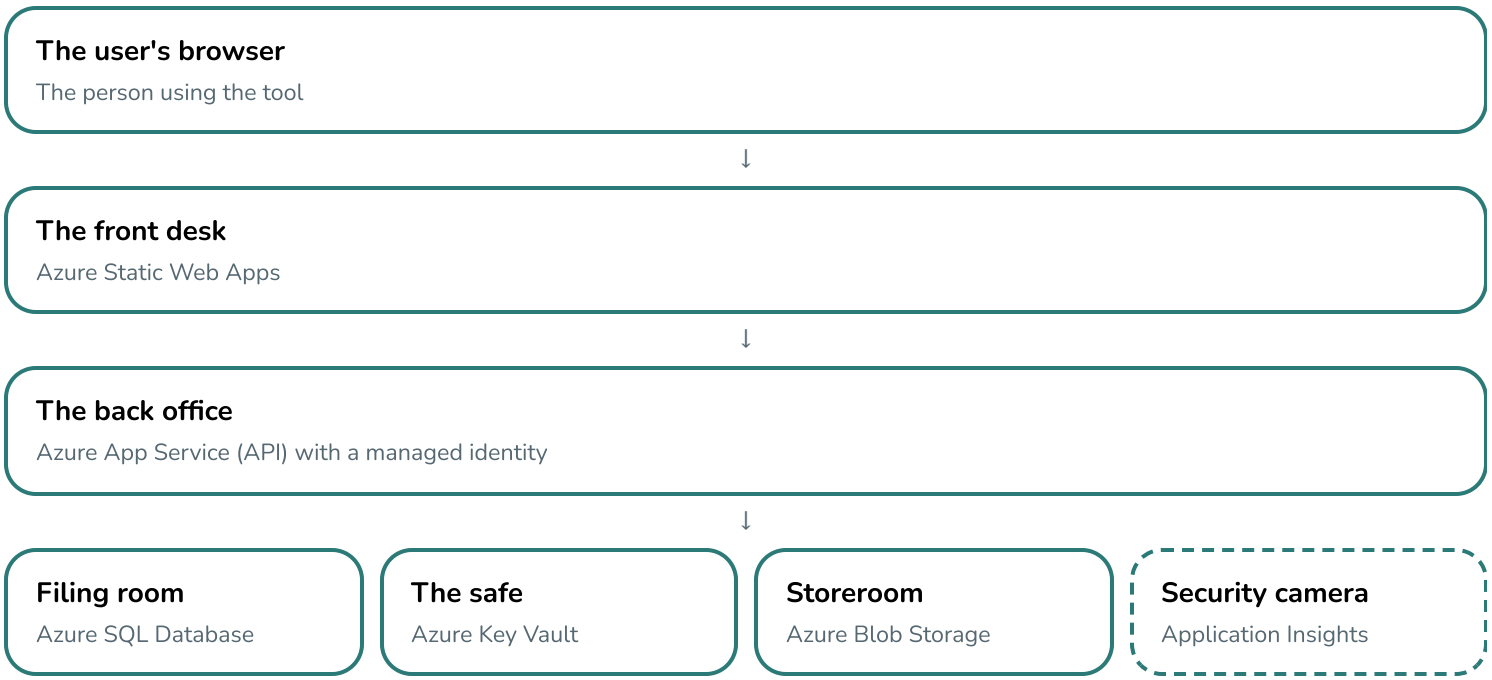
Note what the AI suggested, what you verified and what you changed. It shows your reasoning to the sponsor.

PART 4

How the pieces fit together

A web app is a small team of specialists. Knowing who does what makes talking to IT easier.

The big picture



TECHNICAL DETAIL

Service	POC setting	Notes
Static Web Apps	Free	Built files uploaded with a token
App Service (Linux)	B1, Python 3.12 / Node / .NET	HTTPS only, TLS 1.2, FTPS off, health check, system identity
Azure SQL	S0, Entra-only auth	Optional ledger audit tables
Key Vault	Standard, role-based	Soft delete on
Storage	Standard LRS, private	No public access or shared keys
App Insights	Linked to Log Analytics	Short retention

Keep every environment difference (database address, secrets, allowed origins, demo-data switch) in environment variables read by **one config file**.

Key terms

Term	Plain meaning	Why it matters
Subscription	The billing account	Costs and limits live here
Resource group	A folder for the project	Delete it to remove everything
Region	Which data centre	Often restricted
Managed identity	A staff badge for the app	Opens the filing room and safe with no stealable password
Role	What a badge opens	Give the least needed
Tags	Labels on resources	Cost tracing, ownership

PART 5

Getting access, early

The usual reason a POC slips isn't the building. It's **waiting for permission.**

Three levels of permission

1 Create things

Web app, database, storage. Usually granted for one project folder.

2 Let things talk safely

Allow the app to open the safe without a password. IT guards this.

3 Organisation-wide

Switching on services for the whole subscription. Very few people.

A POC can start with level 1 and fake data. Levels 2 and 3 are needed before anything real.

TIME-LIMITED ACCESS

Some roles last only hours and must be re-activated. If it worked this morning and fails now, re-activate it.

TECHNICAL DETAIL

To do this	You need
Create resources	Contributor on the resource group
Create role assignments	Owner, User Access Administrator or RBAC Administrator
Write Key Vault secrets / blob data	Key Vault Secrets Officer; Storage Blob Data Contributor
Register resource providers	Contributor/Owner at *subscription* level
Microsoft Graph (`az ad`)	Allowed by conditional access

```
az login
az account set --subscription "<id or name>"
az account show -o table      # confirm tenant and subscription first
```

Tools: Azure CLI, Git, Node 20+, your runtime. Open a **new terminal** after installing. If database tokens are blocked on your laptop, use the in-app setup in Part 7. If Graph is blocked, read your object id from the `oid` claim of your ARM token.

Email to IT

Send this on day one. Don't wait until you need the answers.

```
Hello,

We're starting a short proof of concept [one line] using fake data only. Could you help
with:

1. Which subscription and resource group, and which regions are approved?
2. Mandatory tags (cost centre, owner) or a naming standard?
3. Are Azure Static Web Apps allowed, or must the front end sit behind a gateway?
4. Can I create role assignments, or who can for us?
5. Is private networking or a web application firewall required for a POC?
6. Which sign-in method if real users are involved?
7. Any limits on what data a POC may hold?
8. Is my access time-limited? How do I activate it?

Target: demo ready by [date]. Many thanks.
```

PART 6

Building it: the order that works

Foundations first. Each step ends with something a non-technical colleague can review.

Seven steps

1 The records

What's stored and how it relates. Review with the subject lead; hardest to change later.

2 The rule book

Who may do what, in which state. One place the whole system obeys.

3 The history

Every change: who, what, before, after, when. Nothing silently overwritten.

4 Who is who

Simple sign-in for the POC, shaped so a proper one can replace it.

5 The main journey

The thin slice, then other roles.

6 Safety nets

Automatic tests for rules that would quietly corrupt data, including that forbidden actions are refused.

7 Demo data

Invented data covering every state, so any reviewer sees every screen in a minute.

RULES LIVE ON THE SERVER

Hiding a button is courtesy, not security. Anyone can call the system directly, so the system itself must refuse what isn't allowed.

DEPLOY AN EMPTY SKELETON IN WEEK ONE

Permission, region and policy problems show up early, not the night before the demo.

TECHNICAL DETAIL

1. **Schema first** (SQLite locally, Azure SQL in cloud). Portable SQL (`?` placeholders, no `ON CONFLICT` / `MERGE`), one file that knows dialect differences, a test that fails if schema files drift.
2. **Rule table:** `TRANSITIONS: (from, to) → roles` ; `EDITABLE: state → role → fields` . 403 for role problems, 409 for state problems.
3. **Audit:** one row per changed field, in the **same transaction**; write nothing if nothing changed. Append-only (ledger table on Azure SQL).
4. **Auth:** signed, expiring tokens; routes depend only on `current_user` and `require_roles` , so Entra ID can replace the login.
5. **Tests:** rules, permissions (including must-be-refused), calculations. Run on every change.
6. **Seed:** idempotent, gated by `SEED_DEMO_DATA` , only on an empty database.
7. **Operations:** `/health` and `/health/db` from day one; config in environment variables.

PART 7

Putting it on Azure, in two stages

First get something clickable **this week**. Then make it properly secure. Don't skip stage 1; don't mistake it for stage 2.

Two stages

Stage 1 · Demo

A public link to click.

- Fake data only
- Shared simple login
- Data in a file on the web app
- Secrets as settings

For: feedback and first demo.

Stage 2 · Proper

Safe enough for real information.

- Managed database with history
- App signs in with its own identity
- Secrets in a vault
- Monitoring and a tested backup

Needed before: real data or users.

Stage 1: demo

In plain terms: create a project folder, rent a small server for the back office, create a free place for the screens, give the back office its settings, upload the code, build the screens pointing at it, upload them, check the health page says OK, try each role.

TECHNICAL DETAIL

Variables

```
$RG   = "rg-leaveloop-poc"
$LOC  = "<approved region>"
$PLAN = "plan-leaveloop-poc"
$APP  = "app-leaveloop-poc-<unique>"   # becomes <APP>.azurewebsites.net, globally
unique
$SWA  = "swa-leaveloop-poc"
```

1 · Group and plan

```
az group create -n $RG -l $LOC --tags CostCenter=<code> Owner=<you>
az appservice plan create -g $RG -n $PLAN --is-linux --sku B1
```

2 · Static Web App and token

```
az staticwebapp create -g $RG -n $SWA -l <swa region> --sku Free
$swaHost  = az staticwebapp show -g $RG -n $SWA --query defaultHostname -o tsv
$swaToken = az staticwebapp secrets list -g $RG -n $SWA --query properties.apiKey -o
tsv
```

The token is a password. Don't commit it.

3 · The API web app

```
az webapp create -g $RG -p $PLAN -n $APP --runtime "PYTHON:3.12"
az webapp config set -g $RG -n $APP --startup-file "bash startup.sh"
az webapp config appsettings set -g $RG -n $APP --settings `
  SCM_DO_BUILD_DURING_DEPLOYMENT=true ENABLE_ORYX_BUILD=true `
  "SECRET_KEY=<long random string>" "ALLOWED_ORIGINS=https://$swaHost"
```

`ENABLE_ORYX_BUILD=true` makes App Service install requirements during a zip deploy. Without it: 503.

4 · Deploy the API

Zip with code, `requirements.txt` and the startup script at the **root**, forward-slash paths, LF line endings. Build it with a script (Python `zipfile`), not `Compress-Archive`.

```
az webapp deploy -g $RG -n $APP --src-path ./deploy.zip --type zip --async true
# poll https://$APP.azurewebsites.net/health (first start installs packages: a few
minutes)
```

5 · Build and publish the front end

```
cd frontend
$env:VITE_API_URL = "https://$APP.azurewebsites.net/api"   # baked in at build time
npm ci
npm run build
npx --yes @azure/static-web-apps-cli deploy ./dist --deployment-token $swaToken --env
production
```

Anything with a `VITE_` prefix ships to every browser: no secrets. Search the bundle for the host name to confirm it points where you meant.

6 · Verify

- `GET /health` OK; sign in as each role; one allowed and one forbidden action.
- Browser network error but `curl` works: check CORS (`ALLOWED_ORIGINS` must hold the exact front-end URL).

```
az webapp log config -g $RG -n $APP --docker-container-logging filesystem
az webapp log tail   -g $RG -n $APP
```

For a throwaway demo, keep SQLite on the web app's persistent `/home` disk. It doesn't survive scaling to several instances, so move to Azure SQL once the data matters.

Stage 2: proper

What changes: a managed database is created; an administrator sets up its tables once; the app is allowed to read and write rows but **not change the structure**; secrets move to a vault; demo shortcuts are removed.

TECHNICAL DETAIL

- **Entra-only authentication** on the SQL server: no SQL logins exist.
- A named person (ideally a group) is SQL admin.
- The API's **managed identity** is a database user with **read/write on tables only**.
- Optional **ledger** audit tables: `WITH (LEDGER = ON (APPEND_ONLY = ON))` . Tampering becomes *detectable*, not impossible.

One-off table setup

A, from your machine (if developer machines may get database tokens): add your IP to the firewall, `az login --scope https://database.windows.net/.default` , run setup as admin, create the API's user, remove the firewall rule.

B, from inside the web app (when conditional access blocks A):

1. Make the app's identity the **temporary** SQL admin (management call, no database sign-in):

```
az sql server ad-admin create -g $RG -s <sql server> --display-name $APP --object-id <app principal id>
```

2. Run setup at start, controlled by e.g. `SETUP_MODE=real` : create tables, seed reference data, grant the identity read/write. If `FROM EXTERNAL PROVIDER` fails ("Server identity is not configured"), create the user from the identity's **client id** (`CREATE USER [name] WITH SID = ..., TYPE = E`).
3. Watch the setup log until it succeeds.
4. **Restore the human admin**, **delete the setup settings**, restart.
5. Confirm `/health/db` reports the Azure SQL database.

Rehearse on a throwaway database named `*_test` , and make any destructive reset refuse to run elsewhere. Connect with `ActiveDirectoryMSI` on Azure (`ActiveDirectoryDefault` locally). With no server configured, fall back to SQLite.

Secrets and templates

TECHNICAL DETAIL

Stage	Where secrets live
Local	Environment variables or untracked `.env`
Demo, fake data	App settings, typed by the owner, never in the repo
Anything real	Key Vault, referenced from app settings

Reference syntax: `@Microsoft.KeyVault(VaultName=<vault>;SecretName=<name>)` ; the app's identity needs Key Vault Secrets User. **Verify it resolved**, or the app silently uses the literal text as its secret:

```
az rest --method get --url
"https://management.azure.com/subscriptions/<SUB>/resourceGroups/$RG/providers/Microso
ft.Web/sites/$APP/config/configreferences/appsettings?api-version=2022-03-01" --query
"value[].{setting:name,status:properties.status}" -o table
```

If it shows `AccessToKeyVaultDenied` after the role exists, re-run the same deployment (it re-saves settings); a restart may not help.

Infrastructure as code (Bicep)

```
az deployment group what-if -g $RG -f main.bicep -p poc.bicepparam   # lists changes,
makes none
az deployment group create  -g $RG -f main.bicep -p poc.bicepparam
```

- Names from a prefix, the environment and a short hash of the resource group id.
- Same tags on everything via one parameter; no secrets in parameter files.
- Guard role assignments with a switch (`createRoleAssignments`) so it still deploys when you may not create them.
- Look up role ids: `az role definition list --name "Key Vault Secrets User" --query "[].name" -o tsv` .
- What-if can't see quota limits; if a size or region fails, change the parameter and re-run.

PART 8

Keeping it safe

A POC with fake data can be simple. The habits you form now decide how easily it grows up.

Principles

Least access

Everyone and everything gets only what it needs.

No secrets in the open

Never in code, chat, slides or tickets. The owner types it in.

The system enforces the rules

Not the screen.

Keep a record

Every important change: who, what, when.

"TAMPER-EVIDENT"

Like a seal on a jar: it doesn't stop someone opening it, but you can tell if they did. Works best with tight limits on who can write at all.

TECHNICAL DETAIL: POC BASELINE

- HTTPS only, TLS 1.2, FTPS off, health check set.
- Roles enforced on the server by role *and* workflow state; tests for refusals.
- Audit log in the same transaction; ledger on audit tables.
- Managed identity for Azure-to-Azure access.
- Storage private, no shared keys; uploads limited by size and an allow-list (never executables or HTML).
- Front-end build variables are public: no secrets.

Before real data

Item	Why
Company sign-in (and a plan for outside partners)	Replace shared passwords
Secrets in a vault, access by role	No secrets in settings
Database off the public internet	Remove the public path
Edge protection, proper address (by policy)	Defence at the front door
Locked storage for change-history proofs	Makes tamper-evidence credible
Files in private storage, limited-time links	Out of the database; versioned
Cross-region backups, a restore you've tested	Recovery
Owner, runbook, alerts	Someone is responsible
Data classification approved	Compliance

TECHNICAL DETAIL: HARDENING

- Entra ID sign-in (external tenant for partners).
- Private endpoints or restricted SQL firewall; remove "allow Azure services".
- Front Door with WAF; custom domain.
- Time-based immutability policy on the ledger digest container.
- Blob uploads via short-lived links; geo-redundant backups; staging slot (Premium plan).

PART 9

Show it, judge it, decide

A POC exists to be judged. Plan the demo and the decision as carefully as the build.

The demo

1. **The question** in the sponsor's words, and the agreed success measures.
2. **The old way** in 60 seconds, with a real example of the pain.
3. **Walk the thin slice** as each role, in the order a request travels.
4. **Show the history.** Often what wins trust.
5. **Show a refusal:** someone trying something not allowed.
6. **Let a real user drive.**
7. **Finish with the measures,** what's unknown, and your recommendation.

Keep a recording as a fallback.

Decision paper (1 to 2 pages)

- **The answer:** yes, no, or not yet.
- **Evidence** against each success measure, plus users' own words.
- **What running it for real takes:** people, cost range, risks, owner.
- **Options:** stop, extend, pilot, or build for production.
- **Recommendation** and the decision needed, with a date.

TECHNICAL DETAIL: MONITORING AND TEARDOWN

```
requests   | where success == false | summarize count() by name, resultCode,
bin(timestamp, 1h)
exceptions | order by timestamp desc | take 50
```

- Alerts: health check failing, HTTP 5xx, database CPU/DTU over 80%.
- Budget alert set; tags on every resource.
- Teardown: `az group delete -n $RG` removes everything inside. Check the name twice. Key Vault soft delete keeps the vault name reserved for its retention period.

PART 10

When things go wrong

Most problems are common and have known causes.

Symptoms

What you see	Usual reason → what to do
"I can't create this"	Permission or policy limit → ask IT (Part 5)
Worked this morning, "access denied" now	Time-limited access expired → re-activate
Error right after upload	Server still starting or misconfigured → wait a few minutes, check the health page
Screens load, nothing saves	Screens and back office not connected → check address and allowed origins
Bill higher than planned	Something left running → list by cost; scale down or delete
Everyone wants one more feature	Scope creep → "won't this time" list, decision log
Testers don't use it	Wrong problem → back to Part 2, re-interview

Technical fixes

Symptom	Cause and fix
503 after zip deploy	Packages not installed: set `ENABLE_ORYX_BUILD=true`
Startup script fails on Linux	CRLF or backslash paths in the zip: build it with a script
Deploy hangs	Use `--async true`, poll `/health`
Browser "cannot reach server", API fine in curl	CORS mismatch, or SQLite shared across threads (`check_same_thread=False`)
Front end calls the wrong API	Address baked in at build time; old `.env.production` won. Set the variable explicitly; search the bundle for the host
`AuthorizationFailed` on role assignments	Need User Access Administrator or RBAC Administrator
Provider not registered	Subscription-level registration
Region or size not allowed	Policy: pick an allowed one
Key Vault reference not `Resolved`	Missing/stale role: redeploy to re-save settings
No database token on laptop	Conditional access: use in-app setup (Option B)
`az ad` fails	Graph blocked: use the token's `oid` claim
`FROM EXTERNAL PROVIDER` fails	Create the user from the identity's client id
PowerShell breaks `--query` with `[?...]`	Filter the JSON in PowerShell
Rules only in the UI	Anyone can call the API: enforce on the server
History overwritten	Keys ignore the period: fix before real data

PART 11

Checklist and four-week plan

Definition of done

Tick as you go 0 of 16

- [ ] The POC question is written down and agreed by the sponsor
- [ ] Interviews done with each group; findings played back
- [ ] A one-page brief with a "won't this time" list exists
- [ ] Options compared (decision matrix) and the choice explained
- [ ] Cost range checked in the Azure pricing calculator; budget alert set
- [ ] IT questions sent and answers recorded
- [ ] The main journey works end to end for every role
- [ ] The system refuses forbidden actions, and tests prove it
- [ ] Every change appears in the history
- [ ] Demo data covers every state
- [ ] Deployment is scripted; someone else can repeat it
- [ ] Health checks, logs and a budget alert exist
- [ ] Only fake data; no secrets in code or repository
- [ ] Real users tried it; their words are recorded
- [ ] Success measures collected; decision paper written
- [ ] The path to production is costed and has an owner

Four-week plan

Week	Focus	You should have
0	Frame and listen	Brief, interviews, roles and workflow drawn, IT questions sent
1	Foundations	Records, rules, history, tests; empty skeleton on Azure
2	Main journey	Thin slice for every role, demo data, first feedback
3	Proper stage	Managed database, vault, monitoring, insight views
4	Test and decide	Real users, measures collected, decision paper

REFERENCE

Glossary

API

The back office: applies the rules and talks to the data.

Audit log

Record of every important change: who, what, before, after, when.

Azure

Microsoft's cloud: rentable computing, storage and services.

Bicep

A text file describing the cloud resources you want, so they can be recreated.

CORS

Browser rule about which web addresses may talk to the back office.

Entra ID

Microsoft's company sign-in service.

Key Vault

A safe for passwords and keys.

Ledger

Database feature that makes after-the-fact changes to history detectable.

Managed identity

A badge the app itself holds, so no stored password.

MoSCoW

Must, Should, Could, Won't: a way to rank features.

POC

Proof of concept: a small, time-boxed test of whether an idea is worth pursuing.

Resource group

A folder holding everything for one project in Azure.

Role (RBAC)

A named set of permissions for a person or app.

SKU

The size or tier of a service; sets capability and price.

Tag

A label on a resource, such as owner or cost centre.

Thin slice

The smallest complete path through the system, built first.

WAF

Web application firewall: filters harmful traffic before it reaches the app.